

Intermediate (defend one non-obvious trade-off under pushback) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Decision used for this model answer: "I gave every agent access to most of the shared state, instead of giving each agent only its own private slice."

Basic Version

"I chose shared state because the agents genuinely need to see each other's work — the fact-checker can't check facts without seeing what the writer actually wrote, and the writer can't fix things without seeing what the fact-checker flagged.

The strongest objection is: shared state means any agent could accidentally read or even overwrite something it shouldn't touch, and that gets confusing to track as more agents get added.

That's a fair point. What I actually did is a middle ground — most fields are shared and read-only for most agents, but each agent still has one small private field just for its own scratch notes, so it's not fully open, and it's not fully separate either."

This is a complete, honest answer. It states the decision, acknowledges the real cost of the objection, and shows the actual mitigation rather than just brushing the objection off.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

"I designed the shared LangGraph state object as mostly-shared, with each agent also holding a small private scratch field. The alternative — fully private state per agent, passed explicitly between them — is the more conservative, more encapsulated choice, and it's the one a strict software-engineering instinct favors: less surface area for one agent's bug to corrupt another agent's data.

The strongest objection: shared mutable state is a well-known source of hard-to-trace bugs, and it gets worse as agent count grows — this system already has five agents, and any of them could, in principle, write into a field another agent depends on, with no compiler or type system catching the mistake.

What fully private state would have cost me specifically: the fact-checker's entire job is evaluating the writer's draft against the researcher's sources — that's not incidental, it's the core requirement. Fully private state means explicitly passing those fields at every handoff, which is more code, and it doesn't remove the risk the objection raises — it just moves it from 'read the wrong shared field' to 'forget to pass the right field forward,' which is arguably worse because it fails silently (a missing field, not a wrong one) rather than loudly.

What I accepted on purpose: the corruption risk the objection names is real, so I mitigated it directly rather than dismissing it — each agent's write access is scoped by convention and enforced with a lightweight schema check at each node boundary (each agent's node function only writes to its own declared output keys), which gets most of fully-private-state's safety without the handoff overhead.

Verdict: I would not switch to fully private state, because the core requirement (cross-agent visibility for fact-checking) makes the objection's underlying concern unavoidable either way — the real fix is the schema-enforced write boundary, which I already have. I WOULD reconsider the schema-check approach itself if it turned out too permissive in practice — that's a concrete, checkable thing to watch for, not a hypothetical."

Why this answer works: it names the real, non-strawmanned cost of the alternative (encapsulation, less corruption risk) instead of dismissing it. It ties the decision to the specific functional requirement that makes the alternative genuinely worse, not just less convenient. It shows a real mitigation that was actually built, not promised. And the verdict names a concrete condition that would change the answer, which proves the position isn't just stubbornness.

What a weaker answer misses: a weaker answer says "I used shared state because it's simpler" — true, but "simpler for me to write" isn't a defense against "riskier to maintain," it just restates the choice without engaging the objection at all. A weaker answer also sometimes claims the objection doesn't apply ("that would never actually happen") instead of acknowledging the real risk and

showing the mitigation — which reads as either not understanding the objection or being unwilling to admit any cost to your own decision, both of which read poorly under real pushback.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Round 2 — the reviewer doesn't accept the write-boundary mitigation, and attacks it directly:

"Okay, but a write-boundary schema check only catches writes to the wrong field. It does nothing about an agent silently reading a field it has no business depending on — say, the writer reading the fact-checker's private scratch notes and being subtly influenced by them in a way nobody intended. Your mitigation doesn't cover that at all. What then?"

Model response: "You're right, and I should say that plainly instead of dressing it up — the write-boundary schema check was never designed to solve read access, so it genuinely doesn't cover this, it's not a partial fix I'm overstating. Let me separate what it does and doesn't do: it stops an agent from *corrupting* a field another agent depends on, which was the specific risk in the first objection. It says nothing about an agent *reading* something it shouldn't and being quietly influenced by it, which is a real, different risk you've just named correctly.

Is that acceptable as-is, or does it need fixing? For most of the private scratch fields, I'd argue it's a lower-severity risk than the write case — a writer glancing at the fact-checker's scratch notes might bias its next draft in a way that's hard to trace, but it can't silently destroy another agent's output the way an uncontrolled write can. So I'd treat it as a real gap, not a blocking one, and the honest fix is to extend the same node-boundary mechanism to reads: each node declares which state keys it's allowed to *read*, not just write, and that gets schema-checked the same way. I don't have that built today — if you're asking whether the design as it stands actually prevents this, the honest answer is no, and I'd rather say that than invent a justification for why it's fine.

If I were prioritizing what to build first given limited time, I'd build the write boundary before the read boundary, because an uncontrolled write can silently corrupt a field every downstream agent trusts, while an uncontrolled read at worst biases one agent's own output — worse to prevent nothing, but not equally bad to leave unfixed longest."

Curveball — a second decision elsewhere in the design is in tension with this one: *"You just said an uncontrolled write is worse than an uncontrolled read. But earlier you told me your RAG retrieval step writes retrieved chunks straight into shared state with no schema check at all, because 'the retriever is trusted infrastructure, not an agent.' Isn't that the exact risk you just said was the more dangerous one — just exempted because you didn't think of it as an agent?"*

Model response: "That's a real inconsistency, and I'd rather name it than talk around it. I drew the schema-check boundary around 'agents' specifically, and the retriever doesn't call itself an agent, so it fell outside that boundary by default — not because I evaluated its write risk and decided it was safe, but because I was scoping the check by component type instead of by what the check is actually protecting against. The retriever writing malformed or oversized chunks into shared state is

the same category of risk as an agent writing to the wrong field — it just came from a different direction. The fix is to widen the write-boundary check to cover anything that writes into shared state, tool calls and retrieval included, not just agent nodes. I hadn't caught that until you pointed it out just now, and that's a genuine gap in the design as it stands today, not a defensible choice I'm walking back under pressure."

Why this answer works: Round 2 doesn't stretch the original mitigation to cover something it was never built for — it draws an honest boundary around what the fix actually does, names the uncovered risk as real, and gives a prioritization argument for why it wasn't built first, instead of pretending the gap doesn't exist. The curveball response is the harder move: admitting a genuine inconsistency between two decisions, in real time, rather than defending both as if they'd never conflict — which is exactly the difference between a rehearsed defense and someone reasoning about their own design honestly under pressure.

What a weaker answer misses: on Round 2, a weaker answer either claims the write-boundary check "basically" covers reads too (it doesn't, and claiming so is easy to catch), or pivots to a new, invented justification for why read access was never a real risk — both read as unwilling to admit a gap. On the curveball, a weaker answer tries to distinguish the retriever's writes from an agent's writes with a reason invented on the spot ("well, the retriever is different because...") instead of admitting the scoping was inconsistent — which is the same rationalization instinct Doc16's design-self-review technique warns about, just happening live instead of on paper.

Which one should you give, and why?

Open with the Basic version to establish the decision and the honest trade-off in plain terms. If pushed once, move into the Intermediate version's specific mitigation. If pushed a second time — on the mitigation itself, or on a conflicting decision elsewhere — the Advanced version's move is the one that actually matters in a senior review: separate what your fix covers from what it doesn't, concede a genuine gap plainly, and give a real prioritization reason rather than inventing a justification to avoid conceding anything. A design that survives two rounds of honest pushback is worth more than one that sounds airtight after exactly one rehearsed answer.